

Attention and Transformers

Machine Learning (Bilingual)

Fei **Qi** <fred.qi@ieee.org>

Yang **Li**

November, 2023

School of Artificial Intelligence

Xidian University





Word Embedding

Transformer

Vision Transformers

Word Embedding



A word embedding is a learned representation for text where words that have the same meaning have a similar representation.

One of the benefits of using dense and low-dimensional vectors is computational:

- the majority of neural network toolkits do not play well with very high-dimensional, sparse vectors.
- the main benefit of the dense representations is generalization power

¹Neural Network Methods in Natural Language Processing, 2017, p. 92



The bag-of-words (BOW) model is a representation that turns arbitrary text into **fixed-length vectors** by counting how many times each word appears.

This process is often referred to as vectorization.



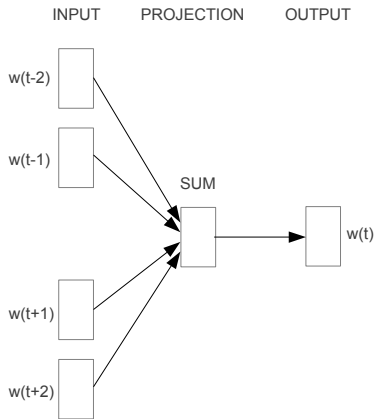
The bag-of-words (BOW) model is a representation that turns arbitrary text into **fixed-length vectors** by counting how many times each word appears.

This process is often referred to as vectorization.

Examples

Document	the	cat	sat	in	hat	with
the cat sat	1	1	1	0	0	0
the cat sat in the hat	2	1	1	1	1	0
the cat with the hat	2	1	0	0	1	1

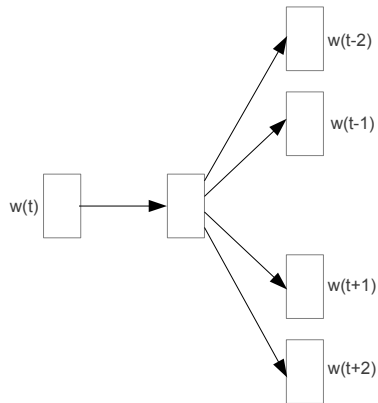
²<https://victorzhou.com/blog/bag-of-words/>

**CBOW**

Word2Vec is a statistical method for efficiently learning a standalone word embedding from a text corpus.

³T. Mikolov et al., ICLR 2013; T. Mikolov, et al. NIPS 2013.

INPUT PROJECTION OUTPUT

**Skip-gram**

Word2Vec is a statistical method for efficiently learning a standalone word embedding from a text corpus.

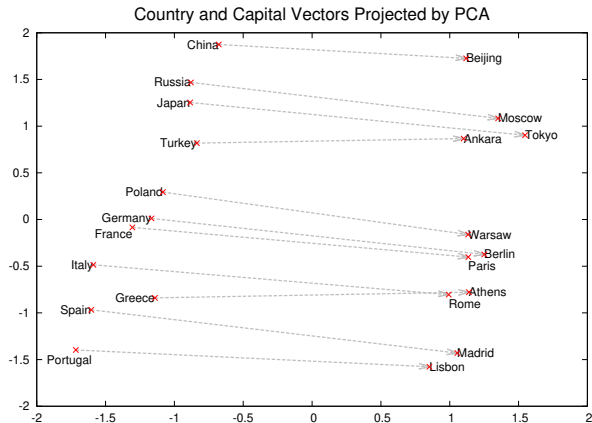
the objective of the Skip-gram model is to maximize the average log probability

$$\frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

The basic Skip-gram formulation defines $p(w_{t+j} | w_t)$ using the softmax function:

$$p(w_O | w_I) = \frac{\exp(v'_{w_O} \top v_{w_I})}{\sum_{w=1}^W \exp(v'_w \top v_{w_I})}$$

⁴T. Mikolov et al., ICLR 2013; T. Mikolov, et al. NIPS 2013.



$$\text{vec}(\text{'Madrid'}) - \text{vec}(\text{'Spain'}) + \text{vec}(\text{'France'})$$

⁵T. Mikolov et al., ICLR 2013; T. Mikolov, et al. NIPS 2013.

Transformer



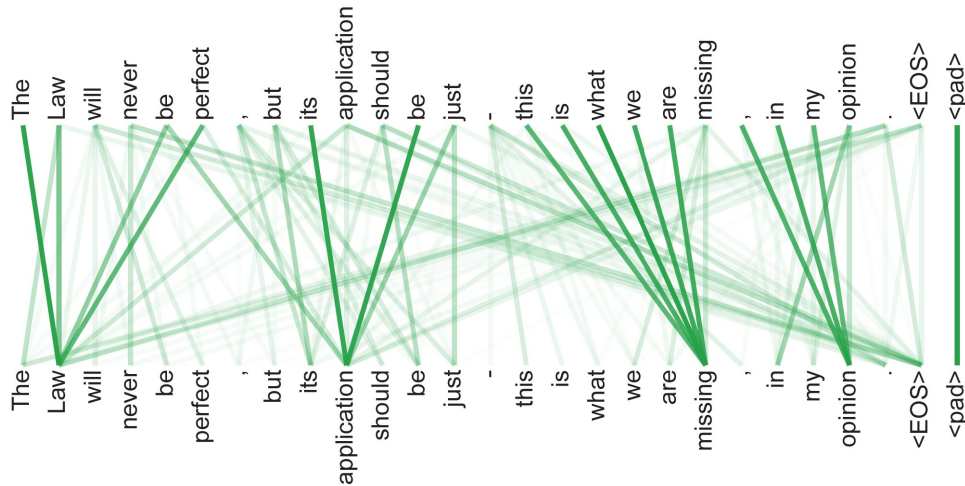
I swam across the river to get to the other bank

I swam across the river to get to the other bank

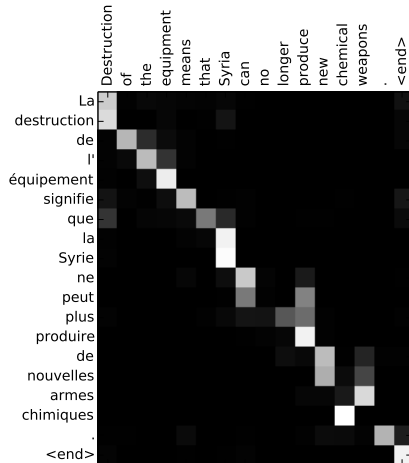
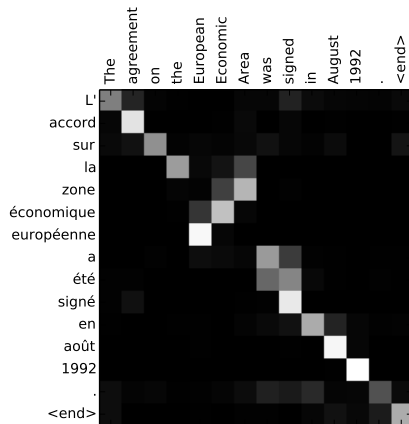
- I swam across the river to get to the other bank.
- I walked across the road to get cash from the bank.

Schematic illustration of attention in which the interpretation of the word 'bank' is influenced by the words 'river' and 'swam'.

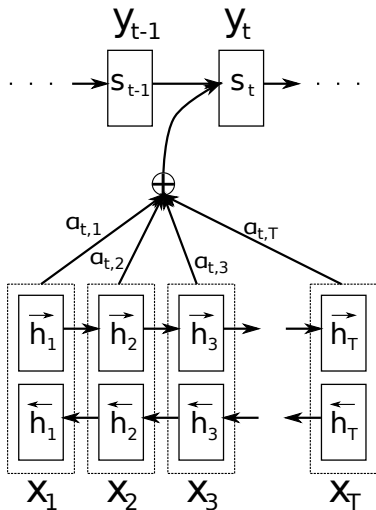
Attention: An example of learned attention weights.



Attention: Joint Learning to Align and Translate⁶



⁶D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR 2015.



The context vector c_i is, then, computed as a weighted sum of these annotations h_j :

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

The weight α_{ij} of each annotation h_j is computed by

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})},$$

where $e_{ij} = a(s_{i-1}, h_j)$ with that:

$$h_t = f(x_t, h_{t-1})$$

$$c = q(\{h_1, \dots, h_{T_x}\}),$$



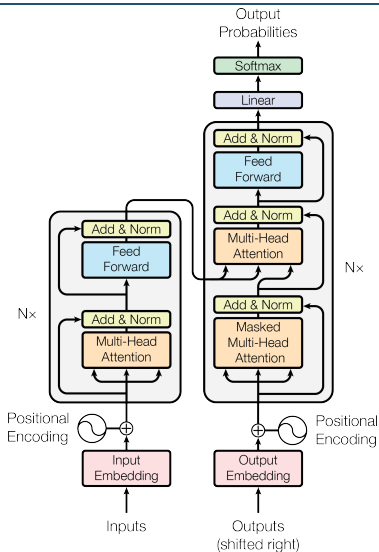
In the Encoder–Decoder framework, an encoder reads the input sentence, a sequence of vectors $\mathbf{x} = (x_1, \dots, x_{T_x})$, into a vector c . Such that:

$$h_t = f(x_t, h_{t-1})$$

$$c = q(\{h_1, \dots, h_{T_x}\}),$$

We obtain an annotation for each word x_j by concatenating the forward hidden state $\vec{h}_j$ and the backward one $\overleftarrow{h}_j$, i.e., $h_j = \left[\vec{h}_j^\top; \overleftarrow{h}_j^\top \right]^\top$.

Attention is all you need (aka. Transformer)



A. Vaswani et al., “**Attention is All you Need**,” NIPS 2017. **Encoder:** $N = 6$ identical layers.

- a multi-head self-attention mechanism,
- position-wise fully connected feed-forward network

Decoder: $N = 6$ identical layers

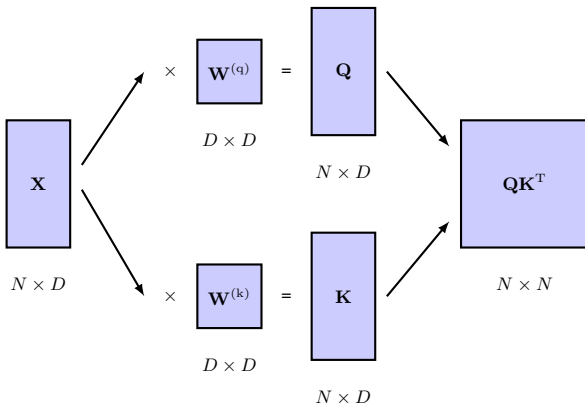
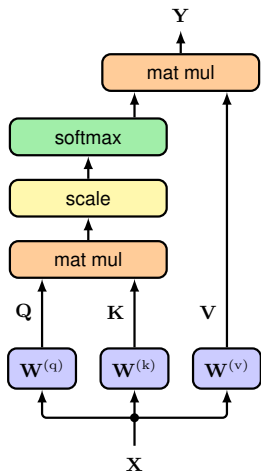
- multi-head attention over the output of the encoder stack

Attention is all you need (aka. Transformer)



The attention is called **"Scaled Dot-Product Attention"**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

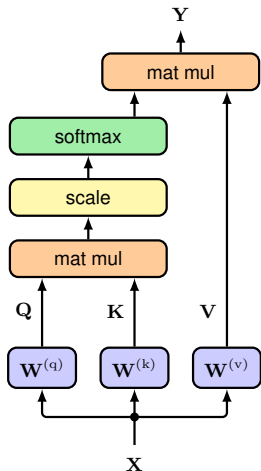


Attention is all you need (aka. Transformer)

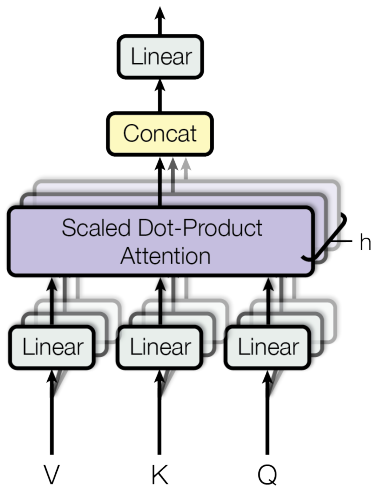


The attention is called “**Scaled Dot-Product Attention**”:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



A matrix diagram illustrating the equation $Y = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$. On the left, a vertical purple rectangle labeled Y has dimensions $N \times D_v$ and a small red square in the top-left corner. In the middle is an equals sign followed by the word 'Softmax'. To the right of 'Softmax' is a large purple rectangle labeled QK^T with dimensions $N \times N$ and a red horizontal bar at the top. This is followed by a multiplication sign $\times$ and another vertical purple rectangle labeled V with dimensions $N \times D_v$ and a red vertical bar on the left.



Multi-Head Attention

we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions.

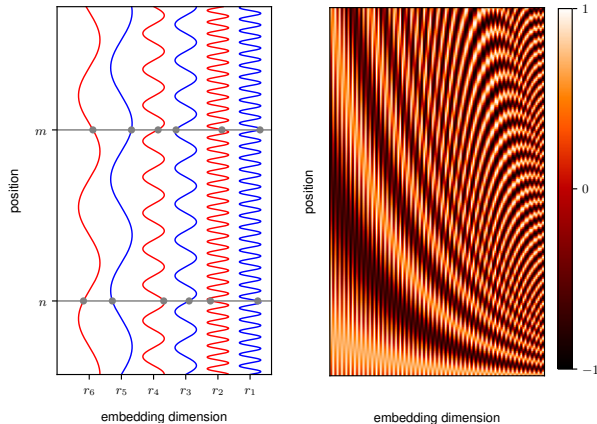


Position-wise Feed-Forward Networks

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Embeddings and Softmax

We use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} .



Positional Encoding

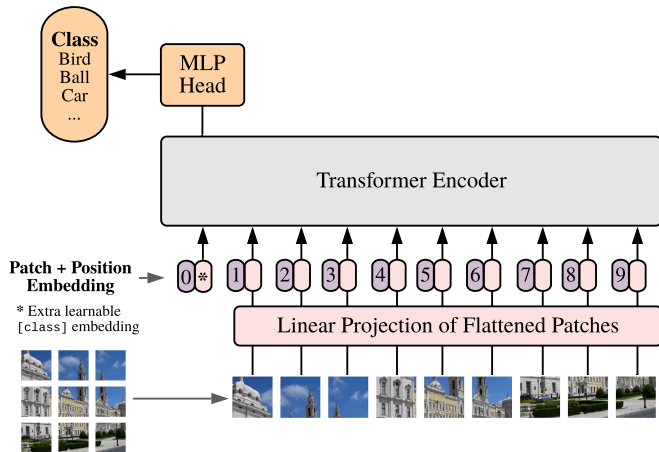
In order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence

$$PE_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d_{\text{model}}})$$

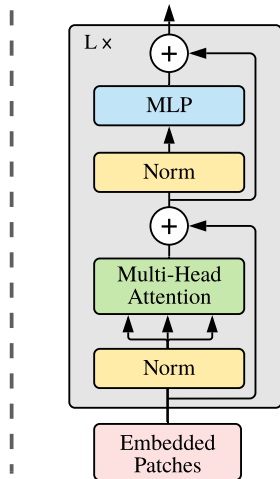
$$PE_{(\text{pos}, 2i+1)} = \cos(\text{pos}/10000^{2i/d_{\text{model}}})$$

Vision Transformers

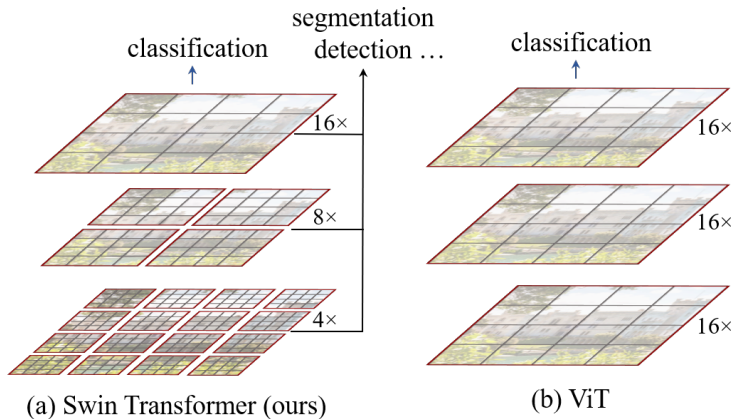
Vision Transformer (ViT)



Transformer Encoder



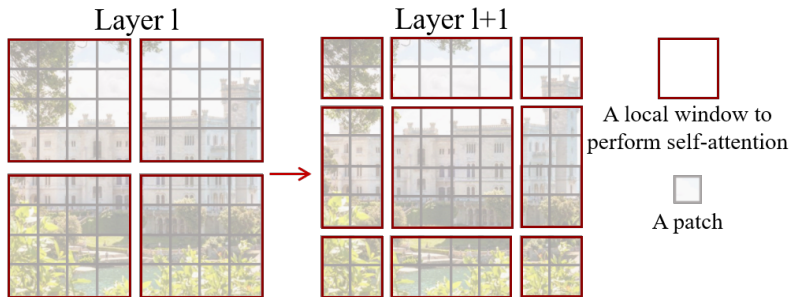
⁷A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," ICLR 2021.



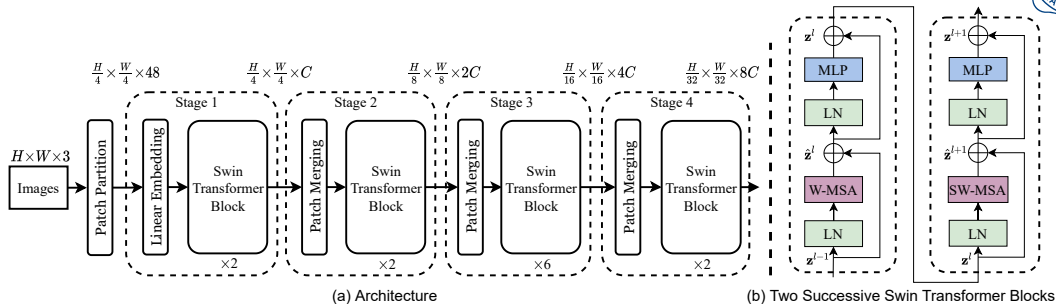
Swin Transformer builds hierarchical feature maps by merging image patches (shown in gray) in deeper layers and has linear computation complexity to input image size due to computation of self-attention only within each local window (shown in red).

ViT produces feature maps of a single low resolution and have quadratic computation complexity to input image size.

⁸Z. Liu et al., ICCV 2021.



An illustration of the shifted window approach for computing self-attention in the proposed Swin Transformer architecture. In layer l (left), a regular window partitioning scheme is adopted, and self-attention is computed within each window. In the next layer $l + 1$ (right), the window partitioning is shifted, resulting in new windows.



- (a) The architecture of a Swin Transformer (Swin-T);
- (b) two successive Swin Transformer Blocks.

W-MSA and SW-MSA are multi-head self attention modules with regular and shifted windowing configurations, respectively.

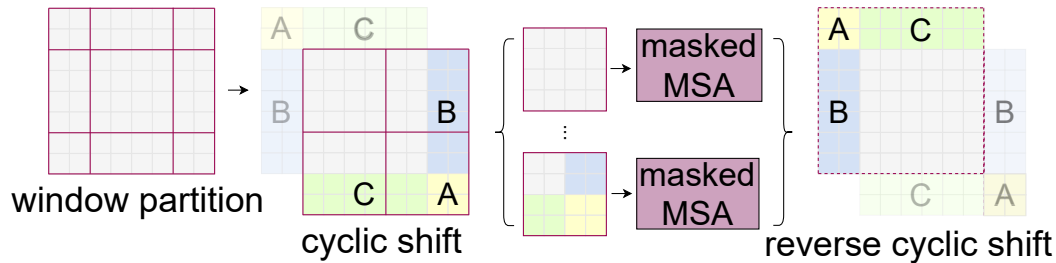


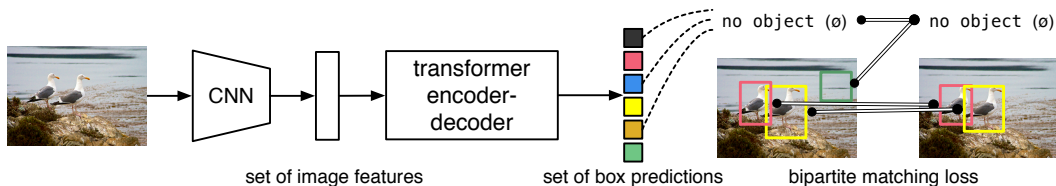
Illustration of an efficient batch computation approach for self-attention in shifted window partitioning.

ImageNet-22K pre-trained models

method	image	#params	FLOPs	throughput	top1 acc
R-101x3	384 ²	388M	204.6G	-	84.4
R-152x4	480 ²	937M	840.5G	-	85.4
ViT-B/16	384 ²	86M	55.4G	85.9	84.0
ViT-L/16	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.4
Swin-L	384 ²	197M	103.9G	42.1	87.3

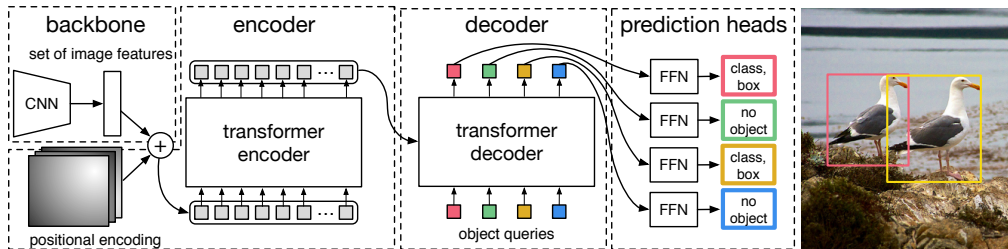
Swin Transformer achieves significantly better speed-accuracy trade-offs:

Swin-B obtains **86.4% top-1** accuracy, which is **2.4%** higher than that of ViT with similar inference throughput and slightly lower FLOPs. Swin-L is **+0.9%** better than Swin-B.

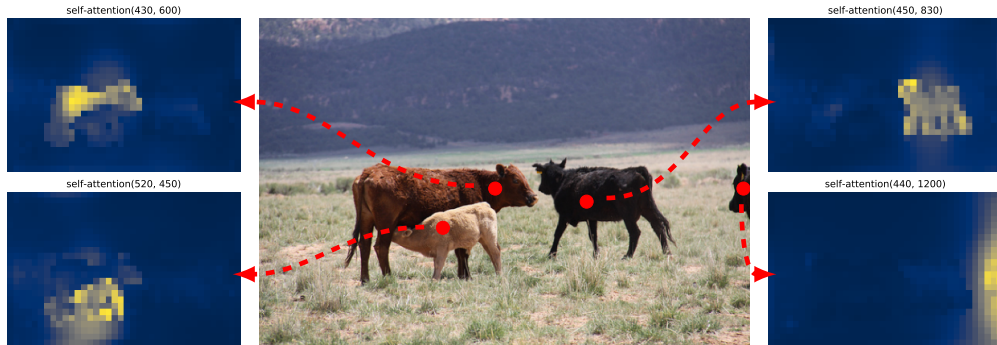


DEtection TRansformer (DETR) directly predicts (in parallel) the final set of detections by combining a common CNN with a transformer architecture. During training, bipartite matching uniquely assigns predictions with ground truth boxes. Prediction with no match should yield a “no object” ($\emptyset$) class prediction.

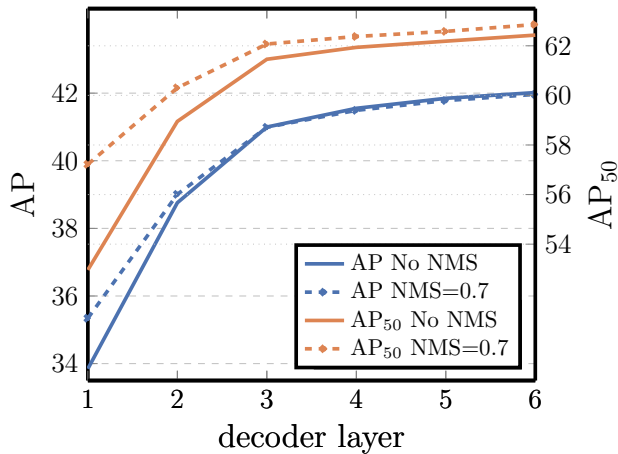
⁹N. Carion, et al., “End-to-End Object Detection with Transformers,” ECCV 2020.



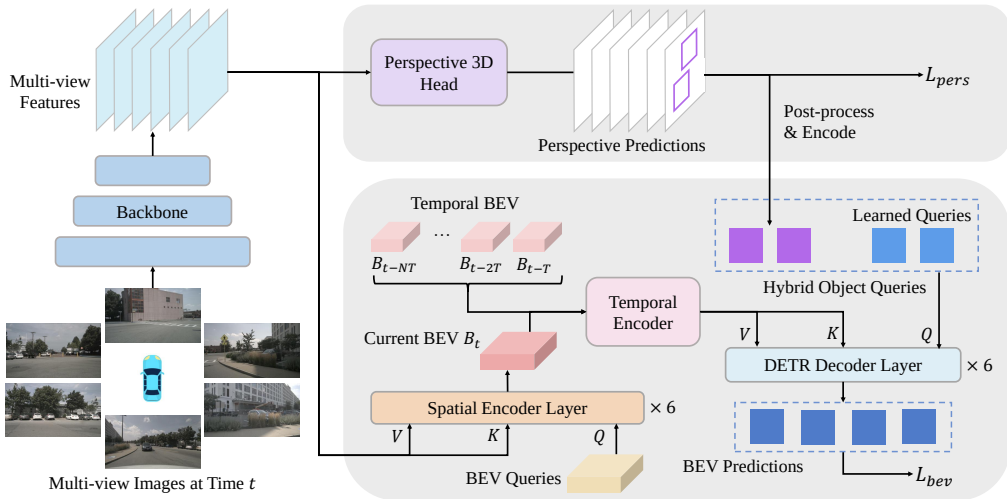
DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call object queries, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a “no object” class.



- Encoder self-attention for a set of reference points.
- The encoder is able to separate individual instances. Prediction made with baseline DETR on a validation image.



AP and AP₅₀ performance after each decoder layer in a long schedule baseline model. DETR does not need NMS by design, which is validated by this figure. NMS lowers AP in the final layers, removing TP predictions, but improves it in the first layers, where DETR does not have the capability to remove double predictions.



¹⁰C. Yang et al., "BEVFormer v2: Adapting Modern Image Backbones to Bird's-Eye-View Recognition via Perspective Supervision," CVPR 2023.